

From empty schema to a live record

VibeLive now collects real environmental measurements on a schedule, at three named locations, with no vendor account, no API key, and no invented numbers. This is what was built, how the parts fit together, and what the data can and cannot be asked to do.

Status **Running** Operating cost **\$0/mo** Free quota used **1.9%** Categories live **2 of 5**

01 THE SHORT VERSION

The asset being built here is **history** — and history is the one thing that cannot be bought back later.

Weather and air quality are free to look up right now, on any phone. What is not free, and not purchasable at any price after the fact, is a continuous, timestamped, per-location record of what conditions actually were. No free provider offers a history endpoint. A month of that record exists only if something was running and writing it down for a month. That is what shipped: the part of the system that writes it down, and keeps writing it down without supervision.

\$0 MONTHLY COST	2 HTTP CALLS / CYCLE	45 READINGS / CYCLE	~2,800 ROWS / DAY
0 API KEYS REQUIRED			

No vendor relationship to manage.

Open-Meteo needs no account, no key, and no card. There is no signup to expire, no key to rotate, no billing surprise. The tradeoff is a non-commercial licence, covered in section 07.

Every number is traceable to a real observation.

Nothing is generated, interpolated, or filled in. Categories with no free source stay visibly empty instead of being padded with plausible curves that would be indistinguishable from real data a month later.

It survives being left alone.

A provider outage backs off instead of hammering. A restart resumes. A duplicate reading is rejected by the database rather than quietly corrupting

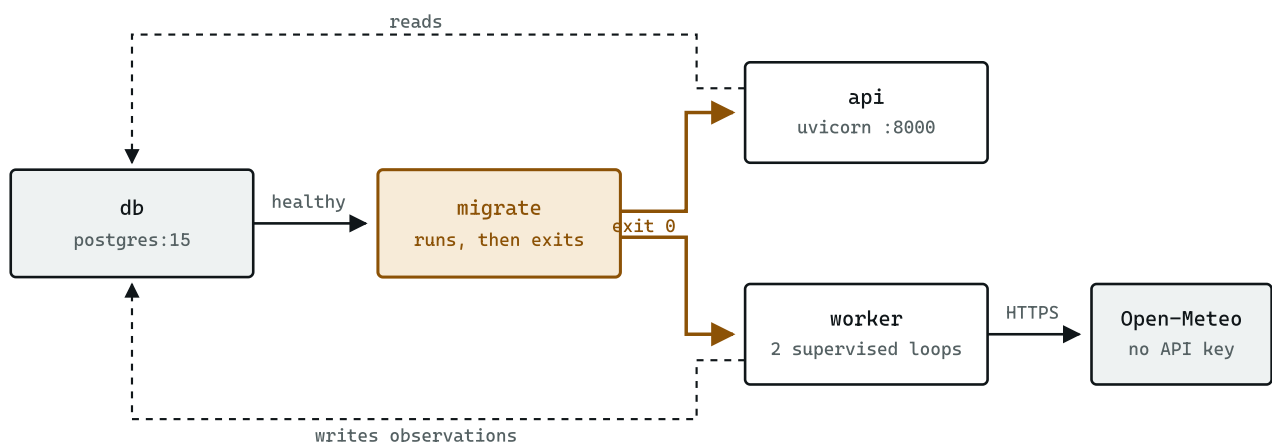
every average computed from it.

WHAT THIS IS NOT, YET

This layer collects. It does not yet analyse, narrate, or display. Trend detection, the REST read surface, and the dashboard are the next phase; the AI narration layer is the one after that. The value of running this now is that those phases arrive to a database with real history in it instead of an empty one.

02 WHERE IT SITS

The stack gained a fourth container. `worker` is a long-lived process, a sibling of `api` rather than a thread inside it, and both are gated on the same migration run.

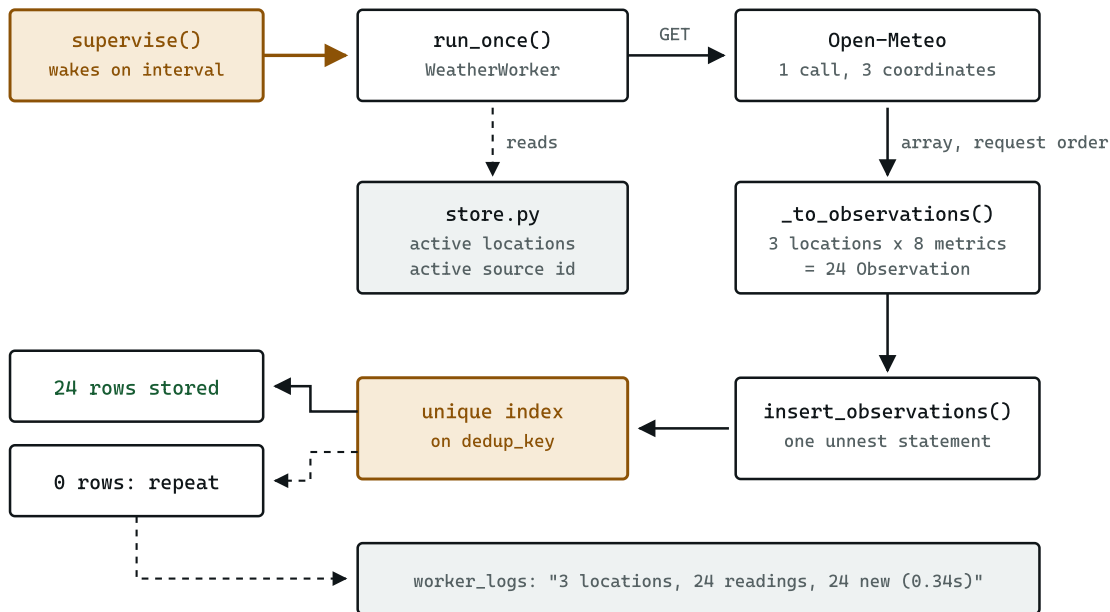


Nothing serves or polls until migrations have finished. Both long-lived containers depend on `migrate` completing successfully, which itself waits for the database healthcheck — so a half-migrated database is unreachable rather than merely unlikely. The worker is the only process making outbound calls.

Separating the worker from the API is a deliberate isolation boundary. A poll that wedges on a slow provider consumes a worker connection and a worker event loop; it cannot starve request handling, because request handling is in another process with its own connection pool.

03 ONE CYCLE, END TO END

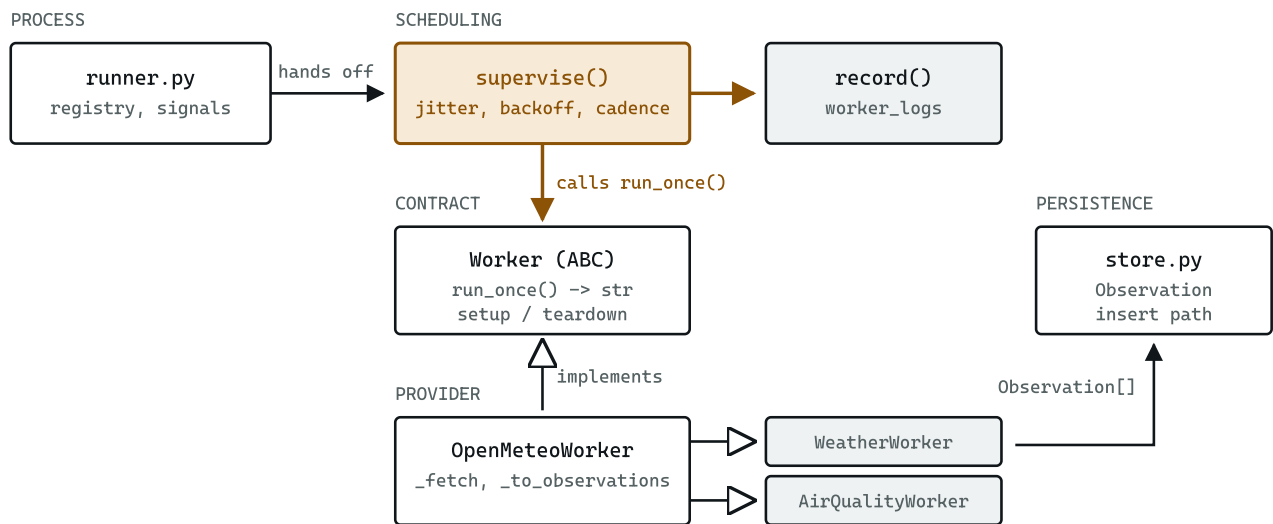
This is the whole mechanism. Everything else in the ingest layer exists to run this reliably, on a schedule, unattended.



All three locations ride in a single HTTP request. Open-Meteo accepts comma-separated coordinate lists and answers with an array in request order, so cycle cost is one call regardless of location count. The array index is the only reliable join back to a location, so a length mismatch is treated as a hard error rather than zipped over. The final gate is the database's own unique index — the application asks to insert 24 rows every time and Postgres decides how many are actually new.

04 THE PARTS, AND WHAT EACH ONE OWNS

The split is deliberate: **scheduling knows nothing about weather, and weather knows nothing about scheduling**. Adding a traffic provider later means writing one class with one method — not touching retry logic, shutdown, logging, or the insert path.



Hollow arrowheads are "is a", solid are "calls". `supervise()` only ever sees the `Worker` contract, which is why it can schedule a future traffic or sensor worker without modification. The two concrete workers at right are configuration, not logic — they set a URL, a category, and a metric map.

Worker

ABSTRACT BASE CLASS

The contract, and deliberately tiny: a `name`, an `interval_seconds`, and an async `run_once()` that does one unit of work and returns a one-line summary. Optional `setup()` and `teardown()` bracket the loop for resources like an HTTP client.

Returning a summary string rather than logging internally is what lets the scheduler own the log format, so every worker's output reads the same in `worker_logs` without any of them agreeing on a convention.

supervise()

SCHEDULER COROUTINE

Wraps any `Worker` in the loop that makes it survivable. It applies **startup jitter** so workers booting together don't fire in lockstep forever, **drift-corrected cadence** so a run that took 40 seconds waits `interval - 40` rather than sliding later every cycle, and **exponential backoff** on consecutive failures, reset by a single success.

It also catches everything a worker can throw. An exception is a logged failure and a longer sleep, never a dead loop — the process does not need a supervisor of its own.

record()

LOG FUNCTION

Appends a line to the `worker_logs` table, and is written so it can never raise. The database is also the thing most likely to be down at the exact moment there is something worth logging, so a failed log write degrades to stdout instead of propagating and killing the worker it was trying to describe.

OpenMeteoWorker

PROVIDER BASE

Everything specific to this provider: build one request carrying every active location, verify the response array length matches, convert the provider's timestamp to an absolute instant, and expand each entry into one `Observation` per metric.

A metric the model has no value for is **skipped**, not stored as zero. Zero is a measurement; absence is not.

WeatherWorker

AirQualityWorker

CONCRETE WORKERS

Pure configuration — a name, a category, a base URL, and a map from the provider's field names to ours. Eight metrics for weather, seven for air quality. No logic, which is the point: a third provider is a similar block of declarations plus one `_fetch`.

Observation

FROZEN DATACLASS

One number, from one source, about one location, at one instant. Immutable by construction. Its most important member is the computed `dedup_key`, built from the instant the **provider** observed rather than the moment we polled — the distinction that section 05 turns on.

store.py

PERSISTENCE MODULE

The only write path into `human_activity`, plus the reference lookups every ingest worker needs. Rows are inserted with a single `unnest` statement — one round trip instead of N, and Postgres reports how many rows were genuinely new, which is what distinguishes "the provider refreshed" from "we polled too fast".

`runner.py`

`PROCESS ENTRYPOINT`

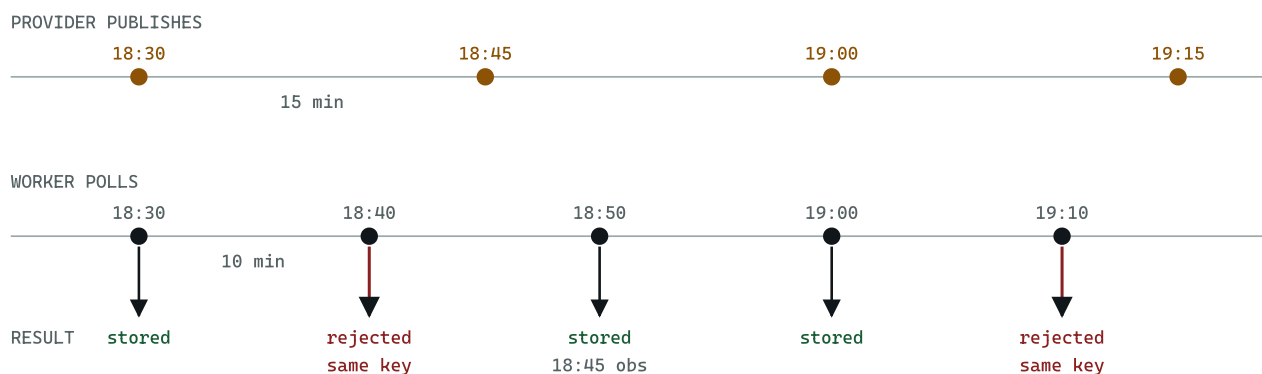
Opens one connection pool, builds workers from a name registry driven by the `ENABLED_WORKERS` setting, and runs them as sibling tasks. An unknown name fails at startup with the list of valid ones rather than silently running nothing.

It installs `SIGTERM` and `SIGINT` handlers that set a shared stop event, so `docker compose stop` lets every worker finish the cycle it is in. Without that, the default is a ten-second wait and then a kill, mid-insert.

05 THREE GUARANTEES, EACH VERIFIED

Polling faster than the source cannot inflate the data

This is the one that would have been invisible. Open-Meteo refreshes weather roughly every fifteen minutes; the worker polls every ten. Some polls therefore return an observation already seen. Stored twice, it counts as two independent samples and biases every average and every trend magnitude computed downstream — with no error, no warning, and no way to detect it afterwards.



The key is the provider's observation instant, not our poll time. A poll at 18:40 receives the 18:30 observation, computes the identical `dedup_key`, and is refused by the unique index. Verified by running two cycles back to back: the second reported `24 readings, 0 new, 24 already stored`.

A source can be switched off at the database

Every worker resolves its source through `active_source_id()`, which returns nothing for a source marked inactive. The worker then does nothing and says so. This is what makes the deactivated `simulated_crowd` and `simulated_noise`

sources structurally unwritable rather than merely unused — the "no invented data" rule is enforced by a column, not by remembering.

It proved itself unprompted during the build: the workers ran once before the migration that activates `open_meteo` had been applied, and logged `source open_meteo is inactive - nothing ingested` instead of writing anything.

Failure decays instead of hammering

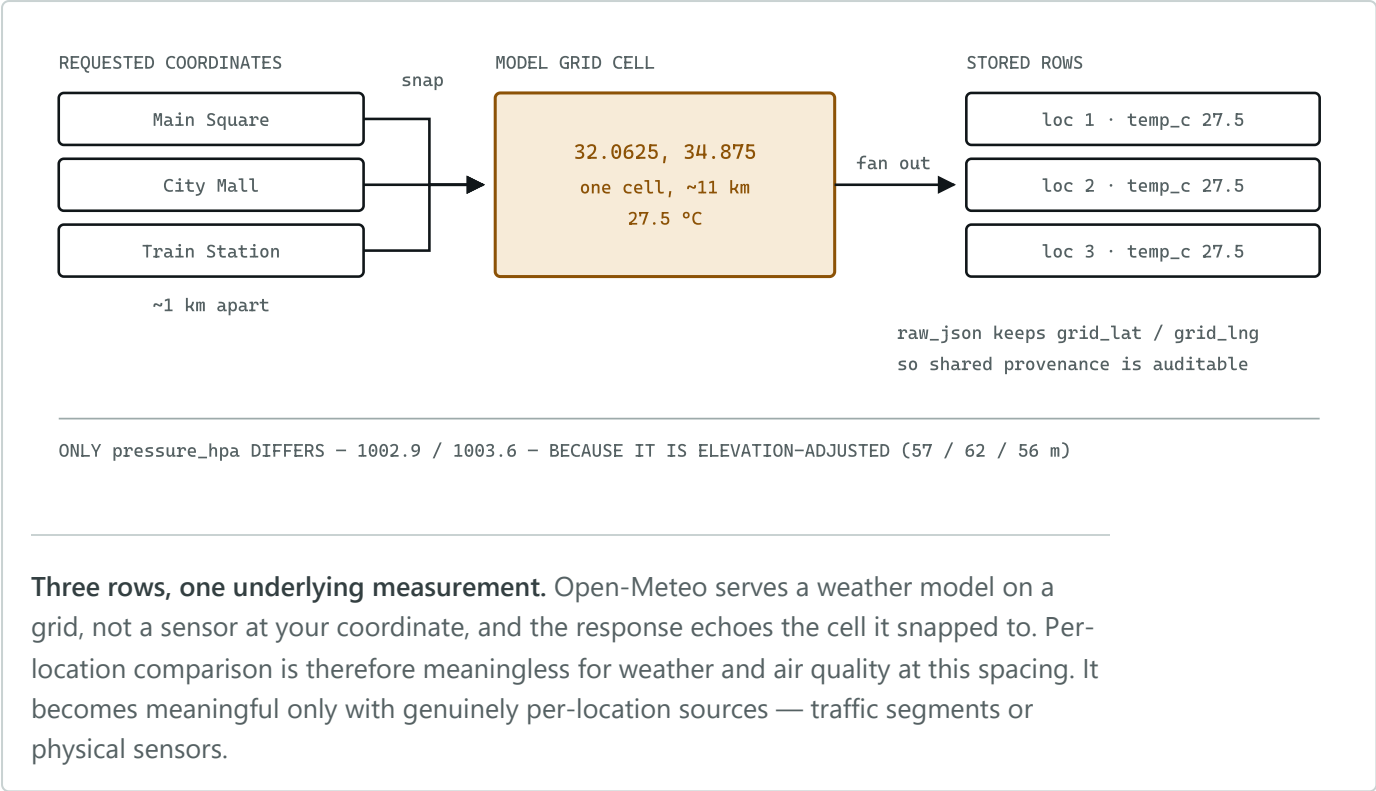
Consecutive failures back off exponentially from the normal interval up to a fifteen-minute ceiling; one success resets the counter to zero. A provider outage therefore costs a handful of requests per hour rather than one every ten minutes forever, and recovery is immediate rather than waiting out a long penalty.

FOUND WHILE COMMITTING

The migration runner fingerprints each migration's **text**. Git on Windows was configured to rewrite line endings on checkout, which changes that fingerprint — so a fresh clone would have failed every migration as "edited after it was applied" when nothing had been edited. A `.gitattributes` pinning the tree to LF now prevents it.

06 WHAT THE DATA HONESTLY IS

The measurements are real. They are also less spatially precise than three separate rows per reading would suggest, and that is worth understanding before anyone builds a conclusion on top of them.



CATEGORY COVERAGE

CATEGORY	SOURCE	METRICS	STATUS
weather	Open-Meteo Forecast	temp, feels-like, humidity, precip, cloud, wind, gusts, pressure	LIVE
pollution	Open-Meteo Air Quality	European AQI, PM2.5, PM10, NO ₂ , SO ₂ , O ₃ , CO	LIVE
traffic	TomTom	Congestion ratio, incident feed	NEEDS FREE KEY
crowd	—	—	EMPTY
noise	—	—	EMPTY

Air quality is hourly at source.

Weather refreshes about every fifteen minutes, air quality once an hour. A thirty-minute pollution window will frequently contain one sample or none, so trend detection on pollution needs a longer window than trend detection on weather.

There is no backfill, ever.

These endpoints serve current conditions only. History exists solely for the period the worker was running. Downtime is a permanent hole in the record, which is why backups matter more here than in a system whose data can be re-fetched.

Crowd and noise are empty on purpose.

Waze is partner-only, Google Places publishes no popular-times endpoint, and ambient noise has no free feed at all. The honest output is a gap, clearly labelled.

07 ECONOMICS

The ingest layer's running cost is zero and its quota headroom is large enough that scale is not the binding constraint — the licence is.

DAILY CONSUMPTION AT THREE LOCATIONS

WORKER	INTERVAL	CALLS / DAY	ROWS / DAY
open_meteo_weather	10 min	144	2,304
open_meteo_pollution	30 min	48	504
total	—	192	2,808
free tier ceiling	—	~10,000	—

That is roughly **1.9% of the daily allowance**, because cost scales with cycles rather than locations — every location rides in the same request. Adding locations costs rows, not calls. Roughly one million rows a year is comfortable for Postgres, but it is the reason a retention policy is on the list rather than an afterthought.

THE REAL CONSTRAINT

Open-Meteo's free tier is **non-commercial**, under CC-BY-4.0. As a personal project this is fine. If VibeLive ever becomes a work product, that licence is a blocker and the switch is to OpenWeather — which is why migration 0010 left the `openweather` source in place and inactive rather than deleting it. Changing provider is then a key, a subclass, and a flag.

08 WHAT IS DELIBERATELY MISSING

Naming these is more useful than a clean bill of health, because every one of them is a decision rather than an oversight.

- **No automated backups.** A dump was taken by hand during the build. Given that history cannot be re-fetched, a scheduled `pg_dump` is the highest-value remaining item in this layer.
- **No retention policy.** At ~2,800 rows a day nothing breaks for a long time, but "a long time" is not a policy.
- **No trend analysis or read API.** The next phase: windowed comparison into `conclusions`, read endpoints, and a dashboard to replace the explorer

redirect.

- **No AI narration.** Every provider costs money, so conclusions will carry `ai_skipped_reason` and the dashboard will render trends computed from the numbers. A local model can fill this in for free later.
- **No authentication, and the SQL explorer is still open.** Correct for a single operator on localhost, unacceptable on any reachable host. `ENABLE_EXPLORER` must be false before this machine is exposed.
- **Not yet pushed to GitHub.** Both commits exist locally only; the remote push needs credentials that are not configured on this machine.